

M5Stack 在庫管理システム

AtomS3 / StickS3 + Unit QRCode

Google Sheets / Apps Script 構築・検証手順

ARCHITECTURE ・ PROTOCOL ・ SETUP ・ ACCEPTANCE TEST



Wi-Fi → Apps Script → Google Sheets

読取りから在庫表までの処理

端末はコードを保持し、Apps Scriptが履歴と在庫表を更新します。



在庫履歴はScansに保存し、Inventoryはその履歴から再集計します。

対応端末・リーダーと事前準備

AtomS3またはStickS3をUnit QRCodeと組み合わせます。



AtomS3

128×128画面 / PORT.A G2・G1



StickS3

135×240画面 / PORT.A G9・G10



Unit QRCode

TRIGボタン / I2Cモード

事前準備

2.4GHz Wi-Fi

Googleアカウント

Google Sheets

VS Code + PlatformIO

Grove / USB-Cケーブル

同じソースをM5Unified / M5GFXで動かし、端末ごとのPORT.Aと画面寸法は実行時に取得します。

Unit QRCodeをI2Cモードで端末へ接続

AtomS3またはStickS3の電源を外してから、PORT.Aへ接続します。



接続前の確認

- 1 端末の電源を外す
- 2 Unit QRCodeの切替をI2Cへ
- 3 端末のPORT.Aへ接続
- 4 接続後にUSB給電

AtomS3: SDA=G2 / SCL=G1

StickS3: SDA=G9 / SCL=G10

ピン番号はソースへ固定しません

TRIGを押して読み取り、SAVEDを確認

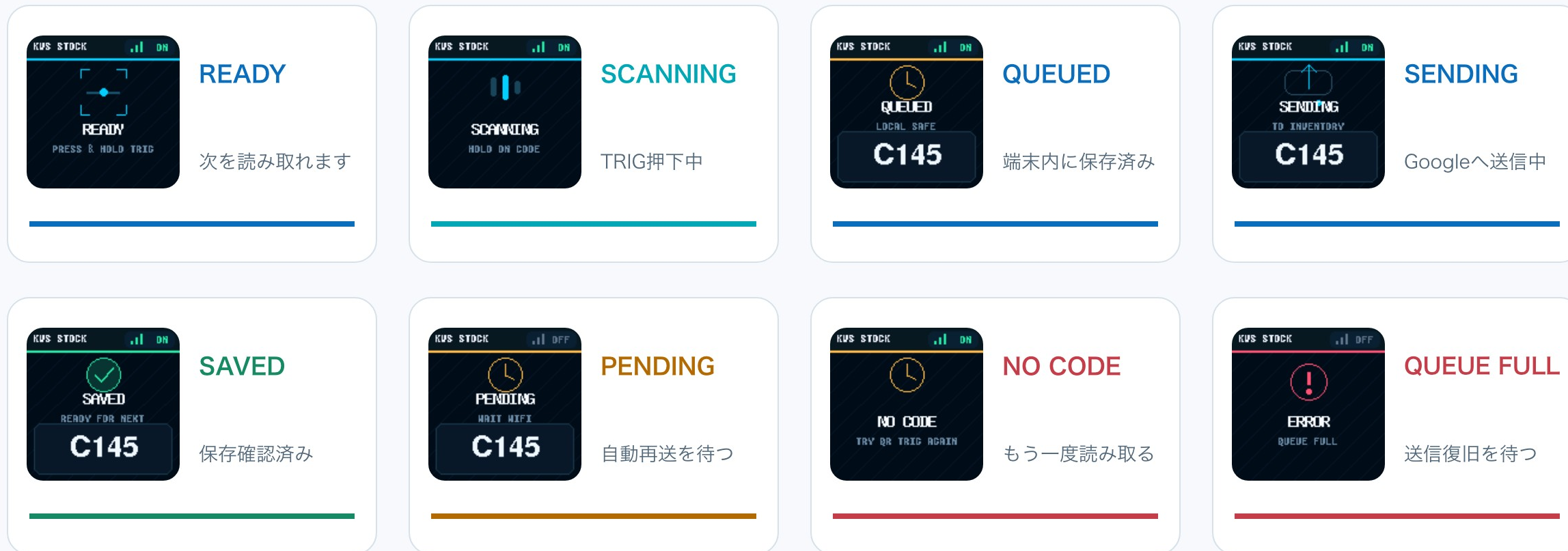
本体側のボタンは使用しません。画面例はAtomS3、状態遷移はStickS3も共通です。



SAVED / READY FOR NEXT が表示された時点で、Apps Scriptによる書込みと読み戻し確認まで完了しています。

画面表示の意味と次の操作

状態名とWi-Fi表示を見れば、待つ・読み直す・設定を確認する判断ができます。



Wi-Fi表示：ON＝接続済み …＝接続中 OFF＝未接続

表示例：AtomS3 / 状態遷移はStickS3も共通

Inventoryシートで数量・状態・画像を確認

ブラウザを閉じていても、Webアプリが受信してデータを追加します。

A	B	C	D	E	F	G	H	I	J	K
コード	保有数	未使用	使用中	廃棄済み	製品名	代表画像	参照元	操作	管理	
A045-B	1	1	0	0	Unit NeoHEX		元画像		RESET INVENTORY Clear Scans + Inventory	
C126	4	4	0	0	M5Stack AtomS3R		元画像		FULL RESET Also clear ProductMaster	
C139	1	1	0	0	PaperS3		元画像			
U173	1	1	0	0	M5Stack Unit QRCode		元画像			

保有数 未使用 + 使用中

状態 未使用・使用中・廃棄済み

商品情報 ProductMasterから表示

操作 状態変更を選択

3シートの役割とデータの流れ

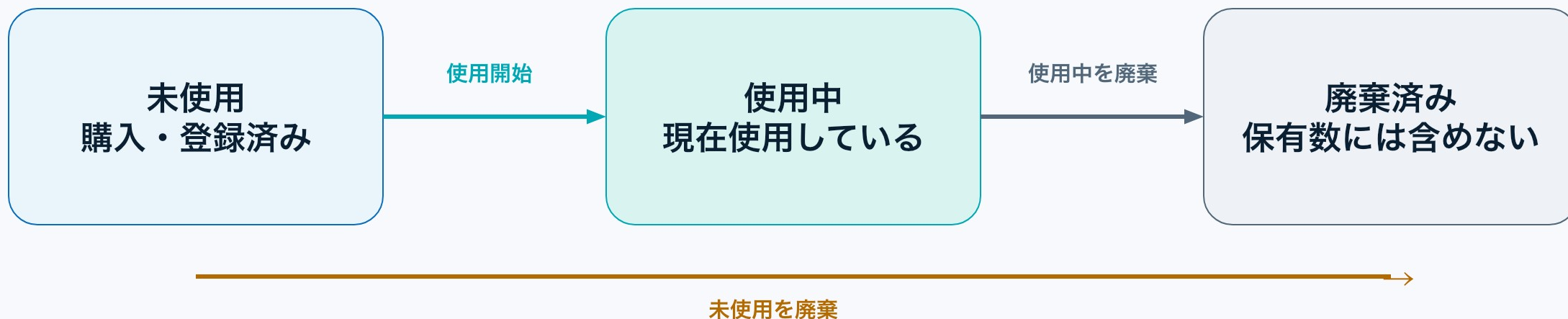
履歴、商品情報、表示用集計を分離しています。



新規コードはScansへ必ず保存。商品検索に失敗しても在庫記録は残り、ProductMasterを後から補完できます。

在庫の状態はInventoryの操作欄から変更

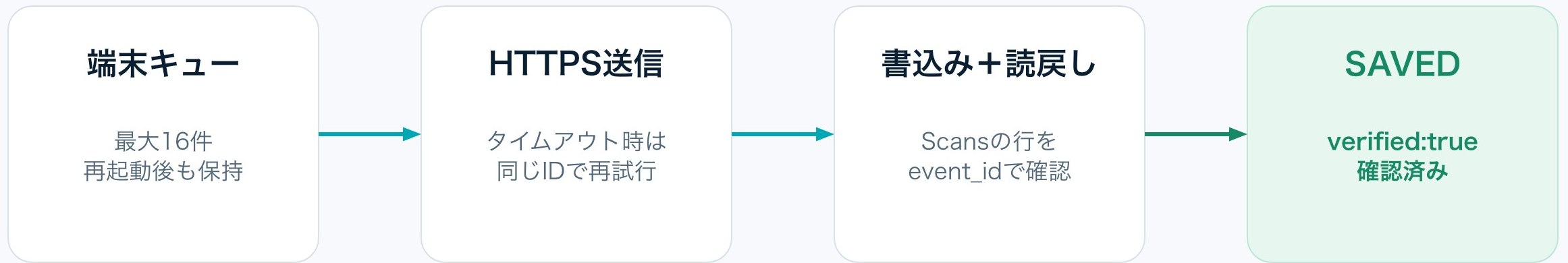
数量セルやScansの履歴は直接編集しません。



すべての状態変更はScansへ履歴として追加されます。
製品名・画像・参照元の修正はProductMasterで行います。

通信が不安定でも二重登録と取りこぼしを防ぐ

同じevent_idを再送し、Google側で重複を判定します。



PENDINGのとき

未確認データは削除されません。Wi-Fi復旧後、同じevent_idで自動再送されます。

SAVEDになったデータだけ端末キューから削除

端末とApps Scriptの受信プロトコル

HTTP 200だけでは成功とせず、JSON応答とeventIdを検証します。

POST /exec application/json

```
{  
  "version": 1,  
  "key": "<DEVICE_KEY>",  
  "eventId": "evt-...",  
  "code": "C145"  
}
```

eventId: 16~80文字の英数字・ハイフン

code: 1~512文字、制御文字なし / 1イベント=+1

保存確認応答

```
{"ok":true, "eventId":"evt-...",  
  "duplicate":false, "verified":true}
```

ok + eventId一致 + verified:true のときだけSAVED

ContentServiceのリダイレクト

302 / 303 → script.googleusercontent.com へGET
共有キーを含むPOSTは転送しない

再試行

BUSY / STORAGE_ERROR / timeout は同じeventId

UNAUTHORIZED / NOT_CONFIGURED / INVALID_REQUEST / SCHEMA_MISMATCH / EVENT_CONFLICT は設定・データを修正

Google SheetsからApps Scriptを開く

空の非公開スプレッドシートを作り、同梱ファイルを配置します。

1 スプレッドシートを作成

M5Stack_Inventory

ファイル名は任意

シートは非公開のまま
使用できます。

2 Apps Scriptを開く

拡張機能

Apps Script

ここから新しいタブを開く

3 2ファイルを配置

Code.gs

apps-script/Code.gs
の全内容を貼り付けて保存

appsscript.json

左の歯車でマニフェスト表示をオン
同梱内容へ置き換えて保存

appsscript.json : Asia/Tokyo / V8 / Sheets・外部取得権限 / 匿名Webアプリ設定

Google画面で「コード.gs」と表示されても問題ありません。

スクリプトプロパティを2件登録し、setupを実行

共有キーはコードへ直接書かず、Google側と端末側へ同じ値を設定します。

1 プロジェクトの設定

スクリプト プロパティ

SPREADSHEET_ID

URLの /d/ と /edit の間
ファイル名やシート名ではありません

DEVICE_KEY

32文字以上のランダムな共有キー
端末側にも同じ値を設定

例: `openssl rand -hex 32`

2 初回セットアップ

setup()を1回実行

- 1 関数選択で setup を選ぶ
- 2 [実行] を押す
- 3 Googleの権限確認を許可
- 4 実行ログの「実行完了」を確認

作成されるシート **Inventory** / **ProductMaster** / **Scans**

OSに合う方法で64文字のDEVICE_KEYを生成

32バイトの暗号的乱数を16進数へ変換します。

macOS / Linux

OpenSSLを利用

```
openssl rand -hex 32
```

OpenSSLが利用できるターミナルで実行

出力例 : a8f3...9c21 (実際は64文字)

Windows 10 / 11

Windows PowerShell 5.1以上

```
$bytes = New-Object byte[] 32
$rng = [Security.Cryptography.RandomNumberGenerator]::Create()
$rng.GetBytes($bytes)
-join ($bytes | ForEach-Object { $_.ToString("x2") })
$rng.Dispose()
```

Windows標準機能のみ。OpenSSLの追加導入は不要

確認 : 0-9とa-fだけの64文字。Web上のランダム生成サイトは使用しない。

生成したDEVICE_KEYを2か所へ同じ値で設定

Google側と端末側が1文字でも違うと認証できません。



確認方法

設定画面とsecrets.hで先頭4文字・末尾4文字を照合。UNAUTHORIZEDなら不一致、SAVEDなら認証と保存確認まで成功。

キー全体をチャット・README・動画へ貼らない

Apps ScriptをWebアプリとしてデプロイ

AtomS3またはStickS3が送信する /exec URLを発行します。

新しいデプロイ

種類

ウェブアプリ

次のユーザーとして実行

自分

アクセスできるユーザー

全員

<https://script.google.com/macros/s/.../exec>

発行後に確認

- 1 /exec で終わるURLを控える
- 2 secrets.h の ENDPOINTへ貼る
- 3 ブラウザで開き死活応答を確認
- 4 コード変更後は新バージョンへ更新

Sheets自体は非公開のままで利用可能

secrets.hにWi-Fi・送信先・証明書を設定

AtomS3とStickS3で同じ設定ファイルを使用します。

platformio/include/secrets.h

```
#define INVENTORY_WIFI_SSID "2.4GHz Wi-FiのSSID"  
#define INVENTORY_WIFI_PASSWORD "Wi-Fiパスワード"  
#define INVENTORY_ENDPOINT "https://script.google.com/macros/s/.../exec"  
#define INVENTORY_DEVICE_KEY "Google側と同じDEVICE_KEY"
```

```
static const char INVENTORY_ROOT_CA[] = R"PEM(  
-----BEGIN CERTIFICATE-----  
... 証明書本文 ...  
-----END CERTIFICATE-----  
)PEM";
```

```
#ifndef INVENTORY_CAPTURE_ONLY  
#define INVENTORY_CAPTURE_ONLY true // 初回確認  
#endif
```

既存の#ifndef～#endifを残す。送信時は *--send-check を選択

SSID / PASSWORD

2.4GHz Wi-Fiの接続情報

ENDPOINT

手順12で発行した /exec URL

DEVICE_KEY

Google側と完全一致させる

ROOT_CA

script.google.com と
script.googleusercontent.com を検証
BEGIN/ENDを含むPEM全文を貼り付けます

secrets.hはGitへ登録・共有しない

ROOT_CAは現在の証明書チェーンを確認して設定

サーバー証明書を固定せず、信頼済みルートCAを照合します。

1 両ホストのチェーンを確認

```
openssl s_client -connect script.google.com:443 \  
-servername script.google.com -showcerts </dev/null
```

```
openssl s_client -connect script.googleusercontent.com:443 \  
-servername script.googleusercontent.com -showcerts </dev/null
```

2 Issuerをたどり、ルートCAを照合

OSの信頼済みCAストア、または `pki.goog/repository/` で
名称・有効期限・SHA-256フィンガープリントを確認

- 1 ルートCAをPEMで取得
- 2 BEGIN / ENDを含めて連結
- 3 INVENTORY_ROOT_CAへ貼付
- 4 実機ログでTLS成功を確認
- 5 期限・Issuer変更時に更新

`setInsecure()` は使用しない

証明書チェーンは更新されるため、公開資料へ特定の古いルート名を固定しません。

接続中の端末に対応する環境を選ぶ

AtomS3とStickS3の書込みコマンドを取り違えないよう、どちらか一方だけを実行します。

リポジトリ直下から実行

```
cd platformio
```

AtomS3を接続した場合 — この1行だけ実行

```
pio run -e atoms3-send-check -t upload
```

StickS3を接続した場合 — この1行だけ実行

```
pio run -e sticks3-send-check -t upload
```

```
pio device monitor -b 115200
```

複数デバイス接続時は `--upload-port` で対象を明示

主要な固定バージョン

```
espressif32: AtomS3 6.9.0 / StickS3 6.12.0  
M5Unified 0.2.22 / M5GFX 0.2.29 / M5UnitQRCode 1.0.1
```

実機で確認

- 1 Wi-Fi ON / READY
- 2 TRIGを押して1件読取
- 3 QUEUED → SENDING
- 4 **SAVED / READY FOR NEXT**
- 5 Scans 1行・Inventory 1件

保存・再送・復旧をテスト用シートで確認

実在庫へ接続する前に、同一イベントが二重加算されないことを検証します。

自動確認

```
node --test tests/ingest.test.cjs
```

```
cd platformio  
pio run -e atoms3 \  
  -e atoms3-send-check \  
  -e sticks3 \  
  -e sticks3-send-check
```

StickS3 : 読取り・Wi-Fi・Sheets送信を実機確認済み

実機シナリオ

- 1 新規ID・新規コード → 数量1
- 2 同じevent_idを再送 → 数量1のまま
- 3 新しいID・同じコード → 数量2
- 4 Wi-Fi切断中に読取 → キューへ保持
- 5 復旧後にSAVED → 1回だけ追加
- 6 Sheetsを閉じてでもWebアプリが記録

合格条件 : SAVEDの件数、Scansの行数、Inventoryの数量がシナリオと一致

異常時の確認順と運用上の境界

履歴を直接削除せず、目的に応じて管理機能を使います。

よくある症状

UNAUTHORIZED	Google側と端末側のDEVICE_KEYを照合
PENDINGが続く	Wi-Fi・ROOT_CA・/exec URL・実行履歴を確認
製品名や画像が空欄	ProductMasterのB～D列と公式SKUページを確認
変更が反映されない	Webアプリを新しいバージョンへ更新
日時がJSTでない	setup()を再実行しタイムゾーンを確認

データの扱い

RESET INVENTORY

ScansとInventoryを初期化
ProductMasterは保持

FULL RESET

3シートすべてを初期化
確認語「完全初期化」が必要

数量・数式・Scansの既存行は
直接編集・削除しない

実機確認：AtomS3 / StickS3ともにUnit QRCode読み取り・Wi-Fi接続・Google Sheets送信を確認済み

制約：個人・少量データ向け / 履歴全体で重複確認 / Workspace匿名公開・クォータ要確認